

UniSENAI JOINVILLE

Curso de Análise e Desenvolvimento de Sistemas (ADS)

Aluna: Ana Paula dos Reis

QUALIDADE DE SOFTWARE

Conceitos, normas e impacto no sucesso dos sistemas

Joinville

Data: 05/07/2026

SUMÁRIO

1 INTRODUÇÃO	3
2 METODOLOGIA	3
3 CONCEITO E EVOLUÇÃO HISTÓRICA DA QUALIDADE DE SOFTWARE	4
3.1 Definições de qualidade segundo os principais autores.....	4
3.2 Da manufatura ao software: uma trajetória histórica.....	5
4 CARACTERÍSTICAS DA QUALIDADE DE SOFTWARE SEGUNDO A ISO/IEC 25010	5
4.1 Adequação Funcional.....	5
4.2 Confiabilidade	6
4.3 Usabilidade.....	6
4.4 Eficiência de Desempenho	6
4.5 Manutenibilidade	6
4.6 Portabilidade.....	6
4.7 Segurança	6
4.8 Compatibilidade.....	7
5 PADRÕES E MODELOS DE MATURIDADE DE PROCESSOS	7
5.1 CMMI (Capability Maturity Model Integration).....	7
5.2 MPS.BR (Melhoria de Processo do Software Brasileiro).....	7
5.3 ISO 9001 e ISO/IEC 15504 (SPICE).....	8
6 QUALIDADE NO PROCESSO DE DESENVOLVIMENTO DE SOFTWARE	8
7 TESTES DE SOFTWARE E GARANTIA DA QUALIDADE.....	9
8 QUALIDADE EM METODOLOGIAS ÁGEIS E DEVOPS	10
9 ESTUDOS DE CASO: CONSEQUÊNCIAS DA AUSÊNCIA DE QUALIDADE	11
9.1 Therac-25	11
9.2 Ariane 5.....	11
10 RESULTADOS E BENEFÍCIOS DA QUALIDADE DE SOFTWARE	12
11 CONCLUSÃO	13
REFERÊNCIAS	13

1 INTRODUÇÃO

Sistemas com falhas frequentes, desempenho instável ou interfaces confusas corroem rapidamente a confiança do usuário e podem custar caro às organizações que os mantêm. Um aplicativo bancário que trava durante uma transferência, um sistema hospitalar que atrasa o acesso a um prontuário ou uma plataforma de comércio eletrônico que perde pedidos em datas de pico ilustram, de forma concreta, o quanto a qualidade determina o sucesso ou o fracasso de um produto de software. Por trás dessas falhas visíveis ao usuário final, existe quase sempre um processo de desenvolvimento que não incorporou, de maneira sistemática, práticas de controle e garantia de qualidade.

Este trabalho parte da premissa de que qualidade não é um atributo acrescentado ao software depois de pronto, mas uma propriedade que resulta de decisões tomadas em cada etapa do ciclo de vida do sistema, do levantamento de requisitos à manutenção em produção. Buscou-se reunir, de forma organizada, os principais conceitos, normas, modelos de maturidade e práticas que sustentam a produção de software confiável, apresentando também casos reais em que a negligência com a qualidade gerou prejuízos financeiros e, em situações extremas, perdas de vidas humanas.

O objetivo geral deste trabalho é analisar como a qualidade de software impacta o sucesso de sistemas de informação, considerando aspectos técnicos, normativos e organizacionais envolvidos em sua obtenção. De forma específica, pretende-se: apresentar o conceito de qualidade de software segundo diferentes autores; descrever as características de qualidade de produto definidas pela norma ISO/IEC 25010; discutir os principais modelos de maturidade de processos adotados pelo mercado; relacionar práticas de desenvolvimento e testes que contribuem para a qualidade; e examinar, por meio de estudos de caso, as consequências da ausência de qualidade em sistemas críticos.

2 METODOLOGIA

A pesquisa caracteriza-se como bibliográfica, de natureza qualitativa e caráter exploratório, desenvolvida a partir da leitura, análise e síntese de obras de referência da engenharia de software, normas técnicas internacionais e artigos científicos disponíveis em repositórios acadêmicos. A pesquisa bibliográfica é desenvolvida a partir de material já elaborado, principalmente livros e artigos científicos, e permite ao pesquisador uma cobertura de fenômenos mais ampla do que aquela que poderia investigar diretamente (NOBRE, 2017).

Foram consultadas obras consolidadas sobre gestão de projetos e qualidade de software, como O'Regan (2025) e Reznick e Monson-Haefel (2012), além de trabalhos acadêmicos disponíveis em repositórios institucionais, a exemplo de Nobre (2017), e artigos técnicos publicados em periódicos e conferências da área, como os disponibilizados pela Association for Computing Machinery (ACM). Recorreu-se ainda à norma ISO/IEC 25010 (2011) como referência normativa central para a caracterização da qualidade de produto de software. A seleção das fontes privilegiou materiais de autoria reconhecida na literatura de engenharia de software, em detrimento de conteúdos sem embasamento técnico ou de caráter estritamente comercial.

3 CONCEITO E EVOLUÇÃO HISTÓRICA DA QUALIDADE DE SOFTWARE

3.1 Definições de qualidade segundo os principais autores

Não existe uma definição única e consensual sobre qualidade de software, mas diversas abordagens contribuem para sua compreensão. Philip Crosby definia qualidade como conformidade aos requisitos, visão objetiva que facilita a mensuração, mas que não distingue produtos igualmente conformes e com níveis distintos de excelência técnica (O'REGAN, 2025). Joseph Juran, por sua vez, descreveu qualidade como adequação ao uso (fitness for use), noção mais ampla, pois reconhece que um produto pode atender formalmente às especificações e, ainda assim, frustrar o usuário se essas especificações não refletirem corretamente suas necessidades reais (O'REGAN, 2025). W. Edwards Deming deslocou o debate para os processos, defendendo que a qualidade do produto final decorre diretamente da qualidade e da estabilidade dos processos que o originam, ideia que fundamenta, até hoje, os modelos de maturidade organizacional discutidos adiante.

De forma mais ampla, pode-se definir qualidade de software como a capacidade de um produto digital satisfazer as necessidades declaradas e implícitas de seus usuários quando utilizado sob condições especificadas (ISO/IEC 25010, 2011). Essa definição é importante por reconhecer que a qualidade não se resume ao cumprimento de requisitos formalmente documentados, mas também às expectativas implícitas dos usuários, como usabilidade, desempenho e confiabilidade. Qualidade não é, portanto, um atributo verificado apenas ao final do desenvolvimento, mas um processo contínuo, planejado e testado ao longo de todo o ciclo de vida do software, envolvendo decisões técnicas, organizacionais e de gestão tomadas por toda a equipe responsável pelo projeto.

3.2 Da manufatura ao software: uma trajetória histórica

No período pré-industrial, o artesão respondia pela integralidade da fabricação de um bem, do projeto à entrega, o que sustentava um forte senso de responsabilidade sobre o resultado final. A industrialização fragmentou essa cadeia produtiva: cada trabalhador passou a responder apenas por uma etapa do processo, o que exigiu a criação de mecanismos formais de controle e inspeção para preservar a qualidade do produto final. Foi nesse contexto que Walter Shewhart desenvolveu, ainda na década de 1920, os fundamentos do controle estatístico de processos, posteriormente ampliados por Deming e Juran (O'REGAN, 2025).

A disciplina de qualidade de software herdou esse arcabouço conceitual a partir da década de 1970, quando o crescimento da complexidade dos sistemas e episódios de falhas de alto impacto evidenciaram que testar o produto apenas ao final do desenvolvimento não bastava para garantir confiabilidade. O caso do chamado "bug do milênio" (Y2K), motivado pela representação de datas com apenas dois dígitos, exigiu correções emergenciais em sistemas legados no mundo inteiro e demonstrou como decisões de projeto aparentemente pequenas podem gerar consequências de grande escala anos depois (O'REGAN, 2025). Watts Humphrey, no Software Engineering Institute (SEI) da Carnegie Mellon University, aplicou os princípios de Deming e Juran especificamente à engenharia de software, dando origem ao campo de melhoria de processos de software (Software Process Improvement), que fundamenta modelos de maturidade como o CMMI, tratado na seção 5 (O'REGAN, 2025).

4 CARACTERÍSTICAS DA QUALIDADE DE SOFTWARE SEGUNDO A ISO/IEC 25010

A norma internacional ISO/IEC 25010, também conhecida como SQuaRE (System and Software Quality Requirements and Evaluation), foi publicada em 2011 em substituição à antiga ISO/IEC 9126 e define um modelo de qualidade de produto de software estruturado em oito características principais, detalhadas a seguir (ISO/IEC 25010, 2011).

4.1 Adequação Funcional

Indica em que grau o software fornece funções que atendem às necessidades declaradas e implícitas dos usuários, quando utilizado sob condições especificadas. Envolve a completude, a corretude e a pertinência funcional das funcionalidades entregues (ISO/IEC 25010, 2011).

4.2 Confiabilidade

Refere-se à capacidade do sistema de manter um nível de desempenho especificado quando utilizado sob condições determinadas, ao longo de um período de tempo. Subdivide-se em maturidade, disponibilidade, tolerância a falhas e recuperabilidade (ISO/IEC 25010, 2011).

4.3 Usabilidade

Indica o grau em que o software pode ser utilizado por usuários específicos para alcançar objetivos determinados, com eficácia, eficiência e satisfação, em um contexto de uso especificado. É um fator determinante para a aceitação do produto pelos usuários finais (ISO/IEC 25010, 2011).

4.4 Eficiência de Desempenho

Diz respeito ao desempenho relativo à quantidade de recursos utilizados nas condições estabelecidas, incluindo tempo de resposta, utilização de recursos computacionais e capacidade de processamento (ISO/IEC 25010, 2011).

4.5 Manutenibilidade

Representa o grau de eficácia e eficiência com que o software pode ser modificado, seja para corrigir defeitos, melhorar o desempenho ou adaptá-lo a um novo ambiente. Está diretamente relacionada à qualidade do código-fonte e à documentação do sistema (ISO/IEC 25010, 2011).

4.6 Portabilidade

Indica a facilidade com que o software pode ser transferido de um ambiente para outro, seja de hardware, software ou organizacional, sem perda de funcionalidade (ISO/IEC 25010, 2011).

4.7 Segurança

Na ISO/IEC 25010, a segurança deixou de ser subcaracterística e passou à condição de característica de qualidade de produto de software, reconhecendo sua importância crescente. Refere-se à capacidade de proteger informações e dados contra acessos não autorizados,

garantindo confidencialidade, integridade, não repúdio, responsabilização e autenticidade (ISO/IEC 25010, 2011).

4.8 Compatibilidade

Indica o grau em que um produto pode trocar informações com outros sistemas e/ou executar suas funções necessárias enquanto compartilha o mesmo ambiente de hardware ou software (ISO/IEC 25010, 2011).

Para cada uma dessas características, a existência de métricas quantificáveis permite que equipes de desenvolvimento avaliem objetivamente o nível de qualidade alcançado, orientando decisões de melhoria contínua ao longo dos ciclos de desenvolvimento.

5 PADRÕES E MODELOS DE MATURIDADE DE PROCESSOS

Além da ISO/IEC 25010, que trata da qualidade do produto de software, existem modelos voltados à maturidade dos processos organizacionais de desenvolvimento, partindo da premissa de que processos mais maduros tendem a gerar produtos de melhor qualidade de forma consistente.

5.1 CMMI (Capability Maturity Model Integration)

Desenvolvido pelo Software Engineering Institute (SEI) da Carnegie Mellon University, o CMMI define níveis de maturidade para os processos de uma organização, variando do Nível 1 (Inicial, processos imprevisíveis e reativos) até o Nível 5 (Otimizado, com foco em melhoria contínua e inovação). O modelo descreve o que os processos devem fazer, e não como devem ser feitos, o que exige que cada organização o interprete e o adapte à própria realidade (O'REGAN, 2025). A certificação CMMI atesta que a organização segue boas práticas reconhecidas internacionalmente na condução de projetos de software.

5.2 MPS.BR (Melhoria de Processo do Software Brasileiro)

Trata-se de iniciativa nacional com objetivo semelhante ao CMMI, porém adaptada à realidade das empresas brasileiras, especialmente pequenas e médias organizações, que muitas vezes não dispõem de recursos financeiros para buscar a certificação CMMI. O MPS.BR também é utilizado como critério em licitações públicas no Brasil (NOBRE, 2017).

5.3 ISO 9001 e ISO/IEC 15504 (SPICE)

A ISO 9001 é norma internacional de gestão da qualidade aplicável a organizações de qualquer setor, incluindo empresas de desenvolvimento de software, que estabelece requisitos para um sistema de gestão da qualidade voltado à melhoria contínua e à satisfação do cliente. Trata-se de abordagem orientada a processos, apoiada em decisões baseadas em evidências e na gestão de relacionamento com as partes interessadas (O'REGAN, 2025). Já a ISO/IEC 15504, também conhecida como SPICE, é um padrão internacional voltado especificamente à avaliação e melhoria de processos de software, bastante utilizado nos setores automotivo e de dispositivos médicos (O'REGAN, 2025).

A adoção desses padrões, embora não seja obrigatória, representa diferencial competitivo relevante, pois demonstra ao mercado o compromisso da organização com a entrega consistente de produtos de qualidade.

6 QUALIDADE NO PROCESSO DE DESENVOLVIMENTO DE SOFTWARE

Garantir a qualidade ao longo do ciclo de vida do software exige a adoção de práticas específicas em cada uma das fases do desenvolvimento. Entre as principais práticas, destacam-se:

- levantamento e documentação clara e completa dos requisitos funcionais e não funcionais, reduzindo ambiguidades que possam comprometer a qualidade do produto final;
- adoção de padrões de codificação (coding standards) que garantam consistência, legibilidade e manutenibilidade do código-fonte;
- revisão de código (code review) realizada por pares, permitindo a identificação precoce de falhas e a disseminação de boas práticas entre a equipe;
- integração e entrega contínuas (CI/CD), com automação de testes a cada alteração no código, reduzindo o risco de regressões;
- refatoração constante do código, eliminando débitos técnicos que possam comprometer a manutenibilidade do sistema ao longo do tempo;
- documentação técnica atualizada, que facilita a manutenção e a evolução do sistema por diferentes equipes ao longo do tempo.

Entre essas práticas, a revisão de código merece destaque histórico. Michael Fagan formalizou, ainda na década de 1970, um processo estruturado de inspeção de código que se mostrou mais eficaz do que os testes isolados na descoberta de defeitos. Estudos posteriores demonstraram que a combinação de testes unitários, de integração e de sistema costuma identificar cerca de metade dos erros presentes em um programa, enquanto a inclusão de revisões sistemáticas de código eleva esse percentual para uma faixa entre 93% e 99% (REZNICK; MONSON-HAEFEL, 2012). Esse dado reforça que testar sozinho não é suficiente: revisar o código lido, e não apenas executado, é parte indispensável da garantia de qualidade.

Essas práticas, quando aplicadas de forma consistente, contribuem diretamente para a melhoria das características de qualidade definidas pela ISO/IEC 25010, resultando em produtos mais confiáveis, seguros e fáceis de manter.

7 TESTES DE SOFTWARE E GARANTIA DA QUALIDADE

Os testes de software constituem uma das principais ferramentas para garantir que o produto atenda aos requisitos especificados e esteja livre de defeitos críticos antes de ser disponibilizado aos usuários. Entre os principais tipos de testes utilizados, destacam-se:

- teste unitário: verifica o funcionamento isolado de componentes individuais do código, como funções e métodos, e é normalmente realizado pelo próprio desenvolvedor, sendo por isso chamado de teste de caixa branca (REZNICK; MONSON-HAEFEL, 2012);
- teste de integração: avalia o funcionamento conjunto de diferentes módulos ou componentes do sistema, com foco nas interfaces entre eles; costuma ser conduzido por uma equipe distinta da que escreveu o código, o que lhe rende a denominação de teste de caixa cinza (REZNICK; MONSON-HAEFEL, 2012);
- teste de sistema: verifica o comportamento do sistema como um todo, em condições semelhantes às de produção, sem conhecimento da implementação interna, o que caracteriza o teste de caixa preta;
- teste de aceitação do usuário (UAT): validado pelo próprio cliente ou usuário final, confirmando que o sistema atende às necessidades de negócio;
- teste de regressão: garante que novas alterações não introduziram falhas em funcionalidades já existentes e previamente testadas;

- teste de desempenho e carga: avalia o comportamento do sistema sob diferentes volumes de utilização, identificando possíveis gargalos.

Além dos testes, a Garantia da Qualidade de Software (Software Quality Assurance – SQA) compreende um conjunto mais amplo de atividades voltadas à prevenção de defeitos, incluindo auditorias de processo, inspeções de código e definição de métricas de qualidade que orientam a tomada de decisão ao longo do projeto. A auditoria de qualidade consiste em uma investigação sistemática sobre a forma como o trabalho é conduzido: o auditor compara o processo efetivamente executado com o processo definido pela organização, relata não conformidades e recomenda ações corretivas, com verificação posterior de sua efetividade (O'REGAN, 2025).

8 QUALIDADE EM METODOLOGIAS ÁGEIS E DEVOPS

A adoção de metodologias ágeis reconfigurou a forma como a qualidade é tratada ao longo do desenvolvimento. Em vez de concentrar os testes em uma fase final do projeto, práticas como o Test-Driven Development (TDD) propõem escrever os casos de teste antes do próprio código: as tentativas de execução falham inicialmente, o código é escrito até que os testes passem, e a suíte de testes é reexecutada a cada nova alteração, garantindo que funcionalidades antigas continuem funcionando (O'REGAN, 2025). O ciclo se repete a cada nova funcionalidade, e a refatoração passa a ser parte natural do processo, e não uma etapa isolada de correção.

A cultura DevOps ampliou esse movimento ao aproximar as equipes de desenvolvimento e de operações, historicamente segmentadas por objetivos distintos. Forsgren, Humble e Kim (2018) demonstraram, com dados coletados de milhares de organizações, que empresas com práticas maduras de integração e entrega contínuas apresentam simultaneamente maior frequência de implantação, menor tempo de recuperação de falhas e menor taxa de mudanças que resultam em incidentes, contrariando a suposição de que velocidade de entrega e estabilidade seriam objetivos concorrentes. A automação de pipelines de build, teste e implantação reduz a intervenção manual, ponto sensível a erros, e permite que problemas sejam detectados minutos após serem introduzidos, e não semanas depois, quando o custo de correção já é significativamente maior.

No contexto acadêmico brasileiro, iniciativas de ensino de DevOps em laboratório têm buscado aproximar estudantes dessas práticas, evidenciando que a apropriação da cultura

DevOps — e não apenas de suas ferramentas — é o principal desafio para equipes que buscam elevar a qualidade de suas entregas (SILVA et al., 2024). Panoramas sobre a adoção de DevOps em organizações brasileiras também apontam maior qualidade percebida do software entregue em produção e maior engajamento entre equipes de desenvolvimento e operação como resultados recorrentes dessa transição (BRAGA, 2015 apud UFPE, 2019).

9 ESTUDOS DE CASO: CONSEQUÊNCIAS DA AUSÊNCIA DE QUALIDADE

A literatura de engenharia de software recorre com frequência a dois episódios ocorridos entre as décadas de 1980 e 1990 para ilustrar, de forma inequívoca, os riscos associados à negligência com a qualidade em sistemas críticos.

9.1 Therac-25

O Therac-25 era um equipamento de radioterapia controlado por software, produzido pela Atomic Energy of Canada Limited (AECL) na primeira metade da década de 1980. Diferentemente de modelos anteriores, a máquina eliminou travas de segurança físicas, transferindo essas funções inteiramente ao software. Uma condição de corrida (race condition) na interface, combinada a mensagens de erro pouco claras e à reutilização de código de versões anteriores sem a adaptação adequada ao novo hardware, permitiu que pacientes recebessem doses de radiação centenas de vezes superiores à prescrita. Entre 1985 e 1987, o equipamento esteve associado a pelo menos seis acidentes graves, com múltiplas mortes (LEVESON; TURNER, 1993).

A investigação conduzida por Leveson e Turner (1993) tornou-se referência obrigatória no ensino de engenharia de software justamente por demonstrar que o problema não residia em um único defeito de código, mas em uma combinação de falhas organizacionais: ausência de revisão independente, documentação insuficiente, testes que não cobriam a sequência exata de operações realizada pelos operadores e uma cultura organizacional que demorou a reconhecer a gravidade dos relatos iniciais.

9.2 Ariane 5

Em 4 de junho de 1996, o foguete europeu Ariane 5 autodestruuiu-se cerca de 37 segundos após o lançamento, resultando em prejuízo estimado superior a 370 milhões de dólares. A causa foi um erro de conversão numérica (overflow) no sistema de referência

inercial: uma variável de velocidade horizontal, armazenada em ponto flutuante de 64 bits, foi convertida para um inteiro de 16 bits sem verificação de limite. O trecho de código havia sido reaproveitado do Ariane 4, foguete mais lento, no qual aquele valor jamais ultrapassava a faixa representável; o Ariane 5, mais veloz, excedeu esse limite ainda na fase inicial do voo (DOWSON, 1997).

Assim como no Therac-25, o problema central não foi a ausência de testes, mas a reutilização de um componente de software fora do contexto para o qual havia sido originalmente validado, sem que o novo cenário de uso fosse submetido a testes específicos em solo (DOWSON, 1997). Os dois episódios, tomados em conjunto, sustentam uma das lições mais repetidas na literatura de qualidade de software: reaproveitar código reduz custo e tempo de desenvolvimento, mas apenas quando acompanhado da revalidação criteriosa das premissas sob as quais aquele código foi originalmente projetado.

10 RESULTADOS E BENEFÍCIOS DA QUALIDADE DE SOFTWARE

Investir em qualidade de software gera benefícios tangíveis tanto para o usuário final quanto para o negócio. Do ponto de vista do usuário, um software de qualidade proporciona maior confiabilidade, menor incidência de erros, melhor experiência de uso e maior segurança na proteção de seus dados pessoais.

Do ponto de vista organizacional, os benefícios incluem a redução de custos de manutenção corretiva, já que a identificação precoce de defeitos é significativamente mais barata do que sua correção após a entrega em produção; o aumento da produtividade das equipes de desenvolvimento, que passam menos tempo corrigindo problemas e mais tempo entregando novas funcionalidades; e o fortalecimento da reputação da organização perante clientes e parceiros, o que pode representar diferencial competitivo relevante no mercado.

Por outro lado, a ausência de qualidade pode gerar consequências severas, como demonstrado pelos casos do Therac-25 e do Ariane 5, apresentados na seção anterior, que resultaram em prejuízos financeiros de grande magnitude e, no primeiro caso, colocaram vidas em risco. Esses episódios reforçam que a qualidade não deve ser tratada como custo adicional, mas como investimento estratégico que se paga ao longo de todo o ciclo de vida do sistema.

11 CONCLUSÃO

A pesquisa desenvolvida ao longo deste trabalho demonstrou que a qualidade de software é fator determinante para o sucesso de qualquer sistema, impactando diretamente a satisfação dos usuários, a sustentabilidade financeira das organizações e a competitividade no mercado de tecnologia. A adoção de normas como a ISO/IEC 25010 fornece um modelo estruturado para definição e avaliação das características de qualidade de um produto, enquanto modelos de maturidade de processos, como o CMMI e o MPS.BR, orientam a melhoria contínua da forma como o software é construído.

Os estudos de caso do Therac-25 e do Ariane 5 tornam esse debate concreto: mostram que falhas de qualidade raramente decorrem de um único erro isolado, mas de uma cadeia de decisões — reutilização de código sem revalidação, testes incompletos, documentação insuficiente — que poderiam ter sido interrompidas em qualquer um de seus elos. Do mesmo modo, a análise das práticas ágeis e da cultura DevOps evidenciou que qualidade e velocidade de entrega não são objetivos necessariamente conflitantes, desde que sustentados por automação de testes, integração contínua e revisão sistemática de código.

Conclui-se que a qualidade não é um ato isolado, realizado apenas na fase de testes, mas um hábito que deve permanecer presente em cada decisão tomada ao longo do ciclo de desenvolvimento, desde o levantamento de requisitos até a manutenção do sistema em produção. Qualidade se constrói em cada decisão, sendo responsabilidade compartilhada por toda a equipe envolvida na construção do software.

REFERÊNCIAS

BRAGA, Fabíola Alves Marinho. Um panorama sobre o uso de práticas DevOps nas indústrias de software. 2015. Trabalho de Conclusão de Curso — Universidade Federal de Pernambuco, Recife, 2015. Disponível em: https://www.cin.ufpe.br/~tg/2019-1/TG_SI/tg_japm.pdf. Acesso em: 05 jul. 2026.

DOWSON, Mark. The Ariane 5 software failure. ACM SIGSOFT Software Engineering Notes, New York, v. 22, n. 2, p. 84, 1997.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: the science of lean software and DevOps: building and scaling high performing technology organizations. Portland: IT Revolution Press, 2018.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO/IEC 25010:2011 — Systems and software engineering: Systems and Software Quality Requirements and Evaluation (SQuARE): system and software quality models. Geneva: ISO, 2011.

LEVESON, Nancy G.; TURNER, Clark S. An investigation of the Therac-25 accidents. Computer, Los Alamitos, v. 26, n. 7, p. 18-41, jul. 1993.

NOBRE, Ivna Diniz. Usabilidade em modelos de maturidade de processo de software e normas de qualidade. 2017. Trabalho de Conclusão de Curso — Universidade Federal do Ceará, Campus Quixadá, 2017. Disponível em: https://repositorio.ufc.br/bitstream/riufc/29555/1/2017_tcc_idnobre.pdf. Acesso em: 05 jul. 2026.

O'REGAN, Gerard. Guide to Software Project Management. 2. ed. Cham: Springer Nature Switzerland, 2025. (Undergraduate Topics in Computer Science).

REZNICK, Alan; MONSON-HAEFEL, Richard. Software Development and Professional Practice. New York: Apress, 2012.

SILVA, J. et al. Integrating DevOps into Software Engineering Education: a laboratory experience. In: BRAZILIAN SYMPOSIUM ON SOFTWARE QUALITY, 23., 2024. Proceedings [...]. New York: ACM, 2024. Disponível em: <https://dl.acm.org/doi/10.1145/3701625.3701688>. Acesso em: 05 jul. 2026.

UNIVERSIDADE FEDERAL DE PERNAMBUCO. Contribuições das práticas DevOps em uma empresa líder do mercado de soluções para apoio à gestão de experiência do cliente: um estudo de caso. Revista de Gestão e Secretariado, São Paulo, 2023. Disponível em: <https://ojs.revistagesec.org.br/secretariado/article/view/1969>. Acesso em: 05 jul. 2026.